

AN1.7 - Supervisor / Orchestrator (A5)

Sprint: AT24 AI Agents - Agent Framework Foundation **Step:** A5 - Supervisor / Orchestrator (the real planner above the A4 runtime) **Depends on:** AF-v1 contracts, Tool Registry (A2), Run persistence applied (A3), Runtime (A4 / G04) **Gate:** G05 - intelligent planning; A4 remains the authority that safely executes.

The Supervisor produces **plan / intent state**. A4's `tick()` still authorizes and executes it, one bounded slice per tick. The LLM, when used, only **proposes** - it never gains tool authority.

1. What was built - `services/agent-framework/supervisor/`

Server-only. Nothing here imports the executor (`invokeTool`); only `plan-proposer.ts` may touch `lib/ai`, via a lazy dynamic import.

File	Role
<code>run-planner.ts</code>	The <code>RunPlanner</code> interface the A4 runtime consumes: <code>plan(definition, input, ctx) -> ordered PlannerToolRequest[] intents + synthesizeOutput(trace, definition) -> structured conclusion.</code>
<code>goal.ts</code>	<code>parseGoal(input)</code> - deterministic, no LLM : extracts <code>symbol / timeframe / question</code> and a cheap <code>isComplex</code> heuristic (multiple sentences, > 28 words, or sequencing markers like "and then" / "compare").
<code>specialist.ts</code>	The <code>Specialist</code> interface - a planning / reasoning role , not a runtime. <code>planTools()</code> (deterministic, within bound tools) + <code>synthesize()</code> (deterministic projection of the persisted trace).
<code>specialists/generic.specialist.ts</code>	Fallback: walk the agent's bound tools in binding order; plain summary synthesis (A4's original behaviour).
<code>specialists/market-intelligence.specialist.ts</code>	Orders <code>market.snapshot -> market.intelligence</code> (only those the agent is bound to); synthesis = a deterministic bullish / bearish / neutral LEAN projected from the pipeline's own regime + hypotheses + <code>intelligenceScore</code> . Explicitly not a signal.
<code>specialist-registry.ts</code>	<code>selectSpecialist(agentType)</code> - <code>MARKET_INTELLIGENCE</code> -> its specialist, everything else -> generic. Adding <code>RESEARCH / STRATEGY_RESEARCH / RISK</code> later is one entry.
<code>plan-shaping.ts</code>	<code>shapeGoalForTool(toolId, goal)</code> - maps the parsed goal into each tool's input shape.
<code>plan-proposer.ts</code>	The LLM boundary : <code>PlanProposer</code> interface, <code>LlmPlanProposer</code> (lazy import("@/lib/ai")), structured-JSON prompt, <code>parseProposedPlan</code> strips fences/prose), and <code>validateProposedPlan()</code> - the governance gate. <code>llmPlanningEnabled()</code> reads <code>AGENT_LLM_PLANNING</code> (default off).
<code>supervisor.ts</code>	<code>SupervisorService</code> implements <code>RunPlanner</code> - deterministic-first, LLM-assist-second.
<code>index.ts</code>	server-only barrel.

1.1 Runtime wiring (minimal)

AgentRuntime gains an optional planner?: RunPlanner dep, **defaulting to a SupervisorService**. tick()'s plan phase calls planner.plan(...); the output phase calls planner.synthesizeOutput(...). No other runtime change - tick(), LimitEnforcer, RunTracer, authorization and the ToolGateway are untouched. The A4 proofs (validate:agent-runtime -> 9/0) still pass with the Supervisor in place.

2. The two planning paths

```
SupervisorService.plan(definition, input, ctx)
|
|-- parseGoal(input)                                deterministic, no LLM
|-- selectSpecialist(definition.type)
|
|-- 1. DETERMINISTIC (always computed)
|   specialist.planTools(def, goal, boundToolIds)
|   -> ordered PlannerToolRequest[] (planningPath: "deterministic")
|
\-- 2. LLM-ASSISTED (only if AGENT_LLM_PLANNING on AND goal.isComplex AND > 1 bound tool)
    proposer.propose({ goal, tools: <bound tools + descriptions> })
    | (LlmPlanProposer: lazy import lib/ai, structured JSON, parseProposedPlan)
    v
    validateProposedPlan(proposal, def, goal, boundToolIds, registry)
    every proposed step must pass ALL of:
      - the agent is BOUND to the tool
      - the tool is in the registry and "active"
      - the agent's PermissionPolicy grants its requiredPermissions
      - the agent's autonomyLevel >= the tool's autonomyFloor
      - not a duplicate
    anything else -> rejected (recorded in planMetadata.llmRejected)
    |
    |-- validated.requests non-empty -> USE IT (planningPath: "llm-assisted")
    \-- empty / proposer threw -> keep the deterministic plan
```

Planning never fails a run. A proposer error or an empty validated result logs and falls through to the deterministic plan.

The LLM never executes. Its output is a JSON list of tool ids it wants ordered; that list is filtered down to the intersection of (*bound* $\wedge$ *registered* $\wedge$ *permitted* $\wedge$ *within autonomy*) before it becomes a single PlannerToolRequest. LLM -> arbitrary tool is structurally impossible.

3. Specialist coordination - not eight engines

Every specialist shares the **one** Agent Runtime, Tool Registry, authorization boundary, evidence model, AgentRun/Step/ToolCall/Evidence persistence, credit accounting and (future) memory. A specialist is ~80 lines: an ordering rule + a deterministic synthesis. The SupervisorService is stateless across ticks - it re-selects the same specialist by definition.type for synthesizeOutput, so a fresh instance reaches the same conclusion.

```

      SupervisorService (RunPlanner)
    /
generic / MARKET_ (RESEARCH, STRATEGY_RESEARCH,
(tool-walk) INTELLIGENCE RISK, ... - later, one entry each)
      \
        |
        | one AgentRuntime.tick()
        | one ToolRegistry / authorizer
        | one evidence + persistence model
```

4. Market Intelligence synthesis - decision support, not a signal

`marketIntelligenceSpecialist.synthesize()` reads the persisted `market.intelligence` tool-call output (a `VerifiedRealTimeIntelligenceContext`) and projects its **already-computed** deterministic fields:

- `bias` = bullish-leaning iff the pipeline's `regime.regimeType` in `{trending-bullish, breakout}` **and** its hypotheses don't contradict; bearish-leaning symmetrically; otherwise neutral.
- `basis` = the regime type + confidence, the bullish/bearish hypothesis tally, and the regime's own `basis[]` strings.
- Always carries: disclaimer ("Decision support only... not a trade recommendation. No entry, stop, target, position size or probability of profit."), `evidenceIds`, `pipelineVersion`.
- An unresolved context -> `bias`: "neutral", `resolved`: false, honest reason.

No LLM. No new market calculation. No second intelligence engine. The validation asserts the output JSON contains **no** `entry` / `stopLoss` / `takeProfit` / `target` / `positionSize` / `signal` key and no `buy` / `sell` / `win-rate` / `probability of profit` language (outside the disclaimer).

5. G05 proof - `npm run validate:agent-supervisor` -> 11 passed, 0 failed

Test	Result
<code>parseGoal</code> extracts symbol; single-instrument analysis is not complex; "and then compare ... run a backtest" is	■
specialist selection: <code>MARKET_INTELLIGENCE</code> -> its specialist, others -> generic	■
deterministic plan (no LLM): MI specialist orders <code>market.snapshot</code> -> <code>market.intelligence</code>	■
deterministic plan: generic specialist walks bound tools in binding order	■
LLM-assist: a valid proposal (reorder of bound tools) is used (<code>planningPath</code> : "llm-assisted")	■
LLM-assist boundary: an unbound tool (<code>backtest.run</code>) in the proposal is rejected -> deterministic fallback	■
LLM-assist: a proposer that throws -> deterministic fallback (<code>planning</code> never fails)	■
<code>validateProposedPlan</code> rejects unbound / unknown / unpermitted / autonomy-floor / duplicate	■
<code>parseProposedPlan</code> strips ``json fences + prose, tolerates garbage	■
structural: only <code>plan-proposer.ts</code> imports <code>lib/ai</code> (lazy); no supervisor file imports the executor; all server-only	■
E2E (real runtime + real services)	■

5.1 The E2E proof (owner's A5 target)

```
startRun({ input: { question: "Analyze XAUUSD and determine whether current
                        conditions support a bullish or bearish setup.",
                        symbol: "XAUUSD" } })
-> SupervisorService.plan -> selectSpecialist("MARKET_INTELLIGENCE")
    plan: [market.snapshot, market.intelligence] (planningPath: deterministic)
-> tick: AgentStep "plan" (planMetadata.specialist = "MARKET_INTELLIGENCE")
-> tick: authorize + invokeTool(market.snapshot) -> MarketDataService [REAL]
-> tick: authorize + invokeTool(market.intelligence)
    -> RealTimeIntelligenceService.build [REAL deterministic pipeline, no LLM]
    -> AgentEvidence x5 (regime + ranked evidence items), provenance intact
-> tick: plan exhausted -> SupervisorService.synthesizeOutput
```

```

        -> marketIntelligenceSpecialist.synthesize(trace)
-> AgentRun.status = "succeeded"
  AgentRun.output = {
    kind: "market-intelligence-conclusion",
    symbol: "XAUUSD", bias: "neutral", resolved: true,
    regimeType: "low-volatility", regimeConfidence: <n>,
    hypothesisCount: <n>, pipelineVersion: "15D.x",
    basis: [...], evidenceIds: [...5],
    disclaimer: "Decision support only. ... No entry, stop, target, ..."
  }

```

Live run this session: XAUUSD: neutral (regime low-volatility), 5 evidence. Governance asserted on the output: no forbidden keys, no signal language. If the provider were down the run terminates `tool_error` with the trace intact (test accepts either).

5.2 Regression

validate:agent-contracts 38/0 - validate:agent-tools 20/0 - validate:agent-run-persistence 17/0 -
 validate:agent-runtime **9/0** (A4 proofs still pass with the Supervisor wired in).

6. TypeScript

`npx tsc --noEmit: 0 errors` in services/agent-framework/**, types/agent-framework/**, scripts/validate-agent-*.ts. Repo-wide 77, all in the stale generated `.next/dev/types/validator.ts` (gitignored, pre-existing).

7. Files changed

Added (10):

```

frontend/services/agent-framework/supervisor/run-planner.ts
frontend/services/agent-framework/supervisor/goal.ts
frontend/services/agent-framework/supervisor/specialist.ts
frontend/services/agent-framework/supervisor/specialists/generic.specialist.ts
frontend/services/agent-framework/supervisor/specialists/market-intelligence.specialist.ts
frontend/services/agent-framework/supervisor/specialist-registry.ts
frontend/services/agent-framework/supervisor/plan-shaping.ts
frontend/services/agent-framework/supervisor/plan-proposer.ts
frontend/services/agent-framework/supervisor/supervisor.ts
frontend/services/agent-framework/supervisor/index.ts
frontend/scripts/validate-agent-supervisor.ts

```

Modified (3):

```

frontend/services/agent-framework/runtime/agent-runtime.ts + planner dep (defaults to SupervisorService);
frontend/services/agent-framework/supervisor/... (n/a)
frontend/package.json + "validate:agent-supervisor"

```

Not touched: Prisma schema, contracts, tool layer, `tick()` / `LimitEnforcer` / `RunTracer` / authorizer internals, legacy services/agents/* + /dashboard/agents. No API routes. No new migration.

8. Non-goals honoured

- No autonomous live trading - no arbitrary LLM tool calling (structural boundary) - no 8 runtime engines (specialists share one runtime) - no agent marketplace - no Agent Builder UI - no credit billing/pricing redesign - no second intelligence engine - no second backtest engine - no unrestricted autonomous loops (`tick()` is still the bounded primitive; `maxSteps`/`maxToolCalls`/`maxRuntimeMs`/`maxCreditCost` still enforced).

- LLM-assisted planning ships **behind AGENT_LLM_PLANNING (default off)** and is only *considered* for complex, multi-tool goals; the deterministic path is always the fallback.

9. Status

A5 complete. The Supervisor plans intelligently (deterministic-first, LLM-assist-second with a hard validation boundary); A4 still safely executes. The E2E goal *"Analyze XAUUSD and determine whether current conditions support a bullish or bearish setup"* runs end to end against the real deterministic intelligence pipeline and produces an evidence-backed, decision-support-only conclusion.

Next per AN1.1 sec 7: A6 (Evidence contract hardening + output-integrity check), A7 (Memory), A8 (Permissions/Guardrails deepening), A9 (Credit ledger), A10 (Evaluation + Observability), then A11-A13 (the three real agents).

G05 review requested.

End AN1.7.